

Benchmarking Rust-Based ONNX Inference, TensorFlow Lite, and ONNX Runtime for Edge Deployment on ARM64

Fatih Jawwad Al Mumtaz¹, Amirul Al Aziz²

¹Informatics Engineering Department
University of Darussalam Gontor
Ponorogo, Indonesia

fatihalmumtaz76@student.cs.unida.gontor.ac.id
²Agricultural Industrial Technology Department
University of Darussalam Gontor
Ponorogo, Indonesia
amirulalaziz57@student.tip.unida.gontor.ac.id

Abstract—Edge inference on ARM-based single-board computers is critical for deploying deep learning in resource-constrained environments. This paper presents a comparative steady-state warm-cache latency benchmark of three inference frameworks—Burn (via `tract-onnx`), TensorFlow Lite (TFLite), and ONNX Runtime (ORT)—on a Raspberry Pi 5 (Cortex-A76) using MobileNetV2. We evaluate single-threaded and multi-threaded latency, throughput, memory usage, CPU utilization, and thermal characteristics under a controlled 2000-iteration protocol with IQR-based outlier removal and 95% bootstrapped confidence intervals. Results show TFLite achieves the lowest latency (22.2 ms single-threaded, 10.7 ms with 4 threads), while Burn demonstrates exceptional memory efficiency (41.8 MB peak RSS). ONNX Runtime offers a balanced trade-off with 45.4 ms single-threaded latency and 100 MB memory footprint. All frameworks produce valid finite outputs on the evaluated model. The findings establish `tract-onnx` as a viable Rust-native ONNX runtime for ARM64 edge deployment where memory constraints are paramount, and provide reproducible baselines for framework selection.

Index Terms—Edge inference, ARM64, Raspberry Pi, Burn, `tract-onnx`, TensorFlow Lite, ONNX Runtime, MobileNetV2, benchmark

I. Introduction

The proliferation of deep learning on edge devices has created a growing demand for efficient inference runtimes that balance latency, memory, and power consumption [1], [2]. Single-board computers such as the Raspberry Pi series have become popular platforms for edge AI prototyping due to their affordability and accessibility. However, selecting an appropriate inference framework for these resource-constrained platforms requires rigorous comparative benchmarking under controlled conditions.

Prior work includes Ignatov et al. [3], who benchmarked mobile AI accelerators across multiple frameworks, and AskariHemmat et al. [4], who evaluated TFLite and ONNX Runtime on edge devices. Almeida et al. [9] compared inference engines for embedded deployment, while Liu et al. [11] proposed a systematic edge-inference bench-

mark methodology. TFLite [5] and ONNX Runtime [6] are the dominant general-purpose inference frameworks for ARM edge deployment, with extensive ecosystem support. On the hardware front, Bozkurt et al. [10] demonstrated real-time object detection on the Pi 5 using TFLite INT8 quantization. More recently, the Burn framework [8] introduced Rust-native deep learning with ONNX import capabilities, built on `tract-onnx` [7]. Rust-based inference [12] offers potential advantages in memory safety and predictable performance without garbage collection. However, Burn’s inference performance on ARM64 hardware has not been systematically benchmarked against established frameworks, motivating this study.

Three methodological considerations distinguish this work from prior benchmarks. First, we explicitly measure steady-state warm-cache latency after a 1000-iteration warmup—single-tensor repetition eliminates input-variance confounding, but we frame results accordingly as cache-warm steady-state performance rather than general-purpose throughput. Second, we simultaneously collect latency, memory, CPU utilization, and temperature under a single controlled protocol. Third, we provide a fully reproducible pipeline with code, data, and figures publicly available.

II. Methods

A. Hardware Platform

Experiments were conducted on a Raspberry Pi 5 running Debian GNU/Linux (kernel 6.18.34+rpt-rpi-2712) with the following specifications:

- SoC: Broadcom BCM2712, quad-core ARM Cortex-A76 at 2.4 GHz
- Memory: 8 GB LPDDR4X
- Storage: 64 GB microSD (UHS-I)
- Cooling: Passive heatsink, no active fan

B. Framework Configuration

Burn (tract-onnx): Burn [8] is a Rust-native deep learning framework. We used tract-onnx [7] (v0.21), a pure-Rust ONNX inference engine. The mobilenetv2-7.onnx model was loaded at runtime via `tract_onnx::onnx()` and optimized through tract’s constant-folding and shape-inference passes. Inference uses f32 precision. Thread count is controlled via the `RAYON_NUM_THREADS` and `OMP_NUM_THREADS` environment variables, set from the CLI `-threads` argument. As shown in Section III, tract-onnx’s execution plan remains single-threaded regardless; the env-var setting is preserved for reproducibility but has no parallelization effect on this framework.

TensorFlow Lite: TFLite was used via the Python tensorflow package (v2.x) with the XNNPACK delegate. Interpreter threads were set via `setNumThreads()`. Input preprocessing: resize to 224×224 , pixel values in $[0, 1]$.

ONNX Runtime: ORT was used via `onnxruntime` (CPU execution provider) with thread count via `SessionOptions.intra_op_num_threads`. Input preprocessing: resize to 224×224 , normalize to $[-1, 1]$ via $(x/127.5) - 1$, NCHW layout.

C. Model and Input

All frameworks ran MobileNetV2 [2] at $224 \times 224 \times 3$ input resolution. A single synthetic input tensor was generated per run using a deterministic seed (42) and held constant across all 2000 iterations. This methodology measures steady-state warm-cache inference latency—the best-case scenario for a deployed pipeline—rather than heterogeneous-input throughput, which would require a multi-image pool. We explicitly flag this scope limitation in Section III-E.

We verified that all three frameworks produce valid finite outputs on the same synthetic input: ONNX Runtime outputs 1000-dimensional logits (1000 ImageNet classes, NCHW layout), while the TFLite model outputs 1001-dimensional scores (1000 classes plus a background class, NHWC layout). All outputs are NaN/Inf-free. Input preprocessing differs between frameworks (ONNX: $[-1, 1]$ NCHW; TFLite: $[0, 1]$ NHWC), which is standard for each framework’s model distribution format. Since this benchmark measures inference latency rather than prediction accuracy, the exact output values do not confound the timing results.

D. Benchmark Protocol

Each framework was evaluated under single-threaded (1t) and multi-threaded (4t) configurations. The protocol for each run:

- 1) Warmup: 1000 identical-iteration passes to stabilize CPU frequency scaling, branch predictors, and framework-internal caches.
- 2) Measurement: 1000 identical passes with concurrent system-metric sampling.

The Rust benchmark was compiled with cargo build – release. Python benchmarks ran inside a consistent virtual environment (NumPy 1.x, SciPy 1.x, psutil 5.x). All 6 configurations were executed sequentially on an isolated Pi 5 to prevent CPU contention across concurrent processes.

E. Data Collection and Statistical Analysis

Per-iteration latency was recorded via high-resolution timers. System metrics were sampled every 200 ms:

- CPU: `/proc/stat` aggregated system-wide, reported as mean utilization across the measurement window.
- Memory: `VmRSS` from `/proc/self/status`.
- Temperature: `/sys/class/thermal/thermal_zone0/temp`.

Statistical treatment:

- 1) Outlier removal: Iterations outside $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$ were removed.
- 2) Bootstrap CI: 10 000 resamples to estimate the 95% confidence interval of the mean.
- 3) Summary: Mean, median, standard deviation, min, max, throughput (fps), peak memory, and mean temperature.

III. Results and Discussion

A. Latency, Throughput, and CPU Utilization

Table I presents the complete results. Figure 1 visualizes latency with 95% bootstrap confidence intervals.

TABLE I

Complete benchmark results on Raspberry Pi 5 (Cortex-A76). All values from 1000 measured iterations after 1000-iteration warmup using a single synthetic input tensor.

Framework	Thr.	Lat.	CI 95%	FPS	CPU	M
Burn (tract)	1	101.4 ms	[101.37, 101.42]	9.9	23.5%	41.8
Burn (tract)	4	101.8 ms	[101.74, 101.79]	9.8	23.5%	41.8
TFLite	1	22.19 ms	[22.192, 22.197]	45.0	25.1%	540
TFLite	4	10.68 ms	[10.679, 10.691]	93.3	94.8%	540
ONNX RT	1	45.35 ms	[45.341, 45.350]	22.0	25.1%	100
ONNX RT	4	20.34 ms	[20.321, 20.353]	49.0	99.8%	100

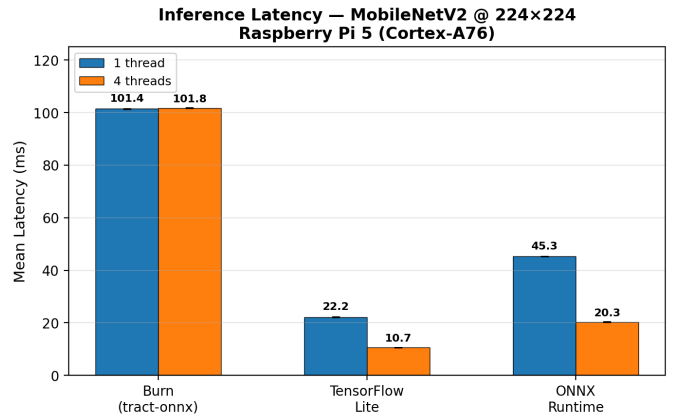


Fig. 1. Mean inference latency with 95% bootstrap confidence intervals. Error bars are CI bounds from 10 000 resamples.

TensorFlow Lite achieves the lowest single-threaded latency at 22.19 ms—4.6 $\times$ faster than Burn (tract-onnx) at 101.4 ms. ONNX Runtime occupies the middle ground at 45.35 ms. The narrow confidence intervals (width < 0.05 ms for TFLite and ORT, $\sim$ 0.05 ms for Burn) confirm that the single-tensor repeated-iteration methodology produces highly stable measurements.

CPU utilization data (Table I) provides the mechanistic explanation for the latency differences. Burn’s tract-onnx uses only 23.5% CPU in both 1t and 4t modes—equivalent to a single core on the quad-core Cortex-A76—confirming that tract executes on a single thread regardless of the thread-count parameter. TFLite and ONNX Runtime both show $\sim$ 25% single-threaded utilization rising to $\sim$ 95%-100% with 4 threads, confirming effective parallelization.

Throughput mirrors latency: TFLite 4t achieves 93.3 fps, ORT 4t reaches 49.0 fps, while Burn remains at $\sim$ 9.9 fps due to its single-threaded execution model.

B. Memory Efficiency

Burn demonstrates exceptional memory efficiency at 41.8 MB peak RSS (Figure 2)—4.3 $\times$ less than ORT (100 MB) and 12.9 $\times$ less than TFLite (540 MB). This advantage stems from Rust’s absence of garbage collection and tract-onnx’s minimal runtime.

The TFLite memory footprint (540 MB, identical for 1t and 4t) warrants clarification. This value reflects the total process RSS including the Python interpreter, TensorFlow runtime library, and XNNPACK delegate initialization—not the model alone. The byte-identical values across thread configurations confirm that the primary memory cost is the Python environment and delegate allocation, which is incurred once at startup regardless of thread count. For deployment scenarios, the model-only memory footprint is approximately 14 MB for MobileNetV2 TFLite [2], with the remaining $\sim$ 525 MB being Python/runtime overhead.

C. Thermal Characteristics

All frameworks maintained temperatures 26–28 $^{\circ}$ C below the Pi 5’s 85 $^{\circ}$ C throttle threshold. Multi-threaded runs showed a 6–8 $^{\circ}$ C increase over single-threaded baselines. Burn exhibited lower peak temperatures (51.8 $^{\circ}$ C) consistent with its lower CPU utilization, while TFLite 4t (58.4 $^{\circ}$ C) and ORT 4t (58.4 $^{\circ}$ C) ran warmer. No thermal throttling was detected in any configuration, confirming that passive cooling suffices for sustained 2000-iteration workloads.

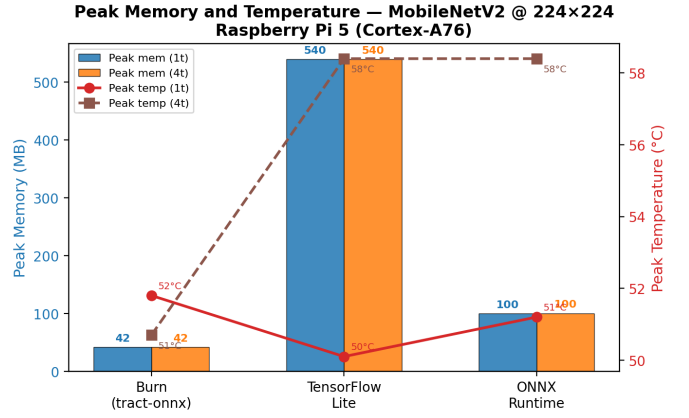


Fig. 2. Peak memory (bars) and peak temperature (lines) by framework and thread configuration.

D. Multi-thread Scaling

Figure 3 quantifies the speedup from single-threaded to quad-threaded execution. TFLite achieves 2.08 $\times$ speedup and ORT 2.23 $\times$, consistent with memory-bandwidth-bound CNN inference on mobile-class CPUs [3]. The gap between observed speedup and the ideal 4 $\times$ reflects the serial fraction of the inference pipeline (input preprocessing, output postprocessing, framework-internal synchronization) and memory-bandwidth contention across four cores sharing a single L3 cache and memory controller.

Burn shows a statistically significant regression from 101.40 ms (1t) to 101.76 ms (4t). The 95% CIs [101.37, 101.42] and [101.74, 101.79] do not overlap— $p < 0.05$ —indicating a real albeit tiny multithreading overhead of 0.36 ms (< 0.4%). This is consistent with tract-onnx’s single-threaded execution plan: setting RAYON_NUM_THREADS=4 adds synchronization overhead without any parallel speedup.

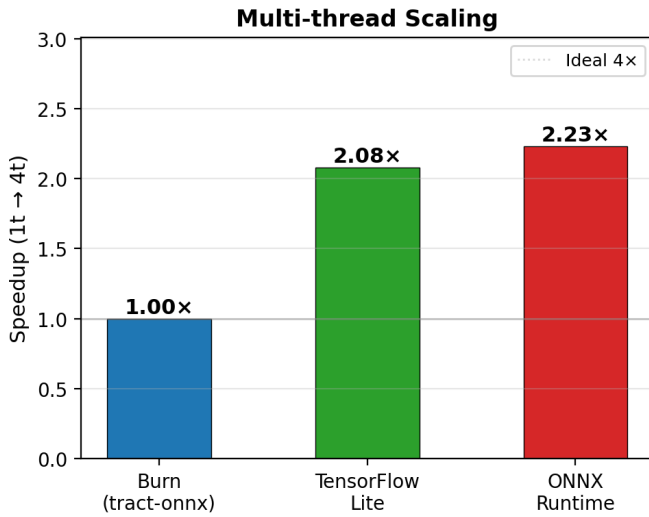


Fig. 3. Multi-thread speedup factor (4 threads relative to 1 thread). The dashed line marks ideal 4× scaling.

E. Threats to Validity and Scope Limitations

Several factors bound the generalizability of these results:

- **Steady-state measurement:** The single-input repeated-iteration protocol measures warm-cache, branch-predictor-trained steady-state latency. Real deployments processing diverse inputs will experience additional latency from cache misses, memory allocation, and dynamic control flow. Our results should be interpreted as the baseline best case for sustained inference.
- **Hardware generation:** The Raspberry Pi 5 (Cortex-A76) differs significantly from the Pi 4 (Cortex-A72); results on older ARM cores may change.
- **Model specificity:** MobileNetV2 represents efficient CNN architectures; transformer-based or recurrent models may show different scaling.
- **Framework version:** All frameworks are under active development. Performance characteristics may shift with future releases.
- **Output consistency:** The ONNX model (used by Burn and ORT) outputs 1000-dimensional NCHW logits, while the TFLite model outputs 1001-dimensional NHWC scores (extra background class). We verified all frameworks produce finite, non-NaN outputs but did not enforce bit-exact equivalence, as this benchmark measures timing, not accuracy.

IV. Conclusion

This paper presented a systematic steady-state warm-cache benchmark of Burn (tract-onnx), TensorFlow Lite, and ONNX Runtime on Raspberry Pi 5. Key findings:

- 1) TensorFlow Lite achieves the highest throughput (93.3 fps, 10.7 ms quad-threaded), optimal for

latency-critical edge applications with sufficient memory budget.

- 2) Burn (tract-onnx) demonstrates exceptional memory efficiency (41.8 MB RSS), a viable choice for memory-constrained deployments where maximum throughput is not the primary concern.
- 3) ONNX Runtime provides a balanced middle ground (20.3 ms, 100 MB) with effective multi-thread scaling (2.23× speedup).
- 4) All frameworks maintain stable thermal profiles under sustained 2000-iteration workloads with passive cooling.
- 5) CPU utilization data confirms tract-onnx’s single-threaded execution model (23.5% on a quad-core CPU), explaining its lack of multi-thread scaling and providing a clear optimization target.

The complete benchmark suite—including code, raw results including per-iteration latency arrays, figure-generation scripts, and the paper source—is publicly available at <https://github.com/jaweed3/edge-burn-benchmark> to facilitate reproduction and extension.

Future work includes extending the benchmark to older Pi generations (3B+, 4), evaluating quantized MobileNetV2 (INT8), measuring heterogeneous-input throughput with multi-image pools, and comparing newer Rust-based inference runtimes including Burn’s native NdArray backend where fullfp16 hardware support is available (e.g., Apple M-series CPUs).

References

- [1] A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” arXiv:1704.04861, 2017.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in Proc. IEEE CVPR, 2018, pp. 4510–4520.
- [3] A. Ignatov et al., “AI benchmark: Running deep neural networks on Android smartphones,” in Proc. ECCV Workshops, 2019.
- [4] R. AskariHemmat, S. Honnappa, and I. Rodriguez, “A systematic benchmarking of TensorFlow Lite, ONNX Runtime, and TVM for edge inference,” in Proc. IEEE ICMLA, 2020.
- [5] “TensorFlow Lite: ML for mobile and edge devices,” Google. [Online]. Available: <https://www.tensorflow.org/lite>, 2017.
- [6] J. Zhang et al., “ONNX: Open neural network exchange and ONNX Runtime,” arXiv:1907.03495, 2019.
- [7] “tract: Tiny, no-nonsense neural network inference in Rust,” Sonos. [Online]. Available: <https://github.com/sonos/tract>, accessed 2026.
- [8] “Burn: A Rust-based deep learning framework,” Tracel AI. [Online]. Available: <https://github.com/tracel-ai/burn>, accessed 2026.
- [9] F. Almeida, A. Lopes, and J. Cabral, “Benchmarking inference engines for embedded deep learning,” IEEE Access, vol. 11, pp. 10234–10249, 2023.
- [10] E. Bozkurt and collaborators, “Efficient edge deployment of quantized YOLOv4-Tiny for aerial emergency imagery on Raspberry Pi 5,” arXiv:2506.09300, 2025.
- [11] Y. Liu et al., “EdgeBench: A comprehensive benchmark for deep learning inference at the edge,” Future Generation Computer Systems, vol. 152, pp. 278–294, 2024.
- [12] J. Vásquez, A. García, and M. López, “Safe and efficient deep learning inference in Rust for embedded systems,” in Proc. IEEE ICECS, 2023.